Architecture

Style: modular monolith — one Next.js 16 deployable (App Router, TypeScript), one Postgres, strict module boundaries inside `src/modules`. Each module can later be extracted into a service because it talks to others only through its exported service functions and owns its tables.

```
src/
  app/[locale]/…        routes only (thin): (site) public, driver, partner-portal, admin
  app/api/…             health, geo, track, driver state/location, event, cron, razorpay webhook
  modules/              business logic (see below)
  shared/lib            env, db, utils, guards (rate-limit/audit), markdown, localized text
  shared/ui|layout|client   design system, shells, client components
  i18n/                 en.ts / hi.ts dictionaries (typed: hi must match en)
prisma/                 schema, migrations, seed
```

Modules and the data they own

Module	Owns	Public surface
`settings`	`SiteSetting` (brand, contact, business/GST, pricing, dispatch, payments, maps, notifications, SEO)	`getSettings/getBrand/updateSetting` (cached by tag)
`auth`	`User`, `Session`, `OtpChallenge`	`requestOtp/verifyOtp/checkOtp`, `getCurrentUser`, role guards
`catalog`	`VehicleType`, `Service`, `RateCard`	cached readers
`pricing`	– (pure engine + quote service)	`computeQuote` (unit-tested), `quoteBooking`
`geo`	`Area` serviceability	provider interface (OSM ⇄ Google), `checkServiceability`
`bookings`	`Booking`, `BookingEvent`, `Review`	`createBooking`, `changeStatus`, `driverAdvance`, state machine
`dispatch`	`DispatchOffer`	`runDispatchRound`, `acceptOffer`, `assignManually`, `dispatchTick`
`providers`	`Company`, `Driver`, `Truck`, `ContractTemplate`, `ProviderContract`	apply, review, e-sign, fleet management
`payments`	`Payment`	cash recording, Razorpay order/verify/webhook
`invoicing`	`Invoice`, `InvoiceCounter`, `LedgerEntry` (written by bookings)	`issueInvoice`
`notifications`	`NotificationLog`	`events.ts` (what to send on each domain event), channels (SMTP, SMS)
`content`	`Page`, `Faq`, `Testimonial`, `Lead`, `Area` (content view)	cached readers, lead action
`analytics`	`AnalyticsEvent`	`trackServer`, event catalogue

Module	Owns	Public surface
`admin`	– (back-office read model + generic resource registry)	allowed to read across modules; writes go through module services

Rules of thumb: pages and actions call module services; modules don't import from `app/`; `shared/` never imports modules (except UI shells that need brand/auth).

Request flow: a booking

```
Customer – /book ─► quoteAction ─► pricing.quoteBooking ─► geo (route + serviceability) + catalog
rate card ─► engine.computeQuote
          ─► createBookingAction ─► bookings.createBooking (re-quotes server-side, PIN, code) ─►
Booking + event
                                                    └─► dispatch.startDispatch ─► offers to N
nearest eligible drivers (+SMS)
Driver  – /driver ─► acceptOfferAction ─► dispatch.assign (atomic updateMany guard) ─► notify
customer, driver, **company**
        ─► driverAdvanceAction ─► EN_ROUTE → ARRIVED → (PIN) IN_PROGRESS → COMPLETED
                                              └─ completion: payment record (cash), GST invoice,
ledger entry, emails
Cron /api/cron/tick ─► expire offers, next round / NO_DRIVER_FOUND, release scheduled bookings,
cleanups
```

Booking state machine (`modules/bookings/state.ts`)

`PENDING_DISPATCH → OFFERED → ASSIGNED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED`; `CANCELLED` until work starts; `NO_DRIVER_FOUND` escalates to dispatchers. Every transition goes through `canTransition()` and writes a `BookingEvent` (audit trail for support).

Partner onboarding & contracts

`Apply (driver|company)` → `PENDING` → admin **approve** (sets commission, renders the active `ContractTemplate` into an immutable `ProviderContract.renderedBody`, emails the partner) → partner **e-signs** (typed name + email OTP; time/IP/method stored) → `ACTIVE` (company drivers activate with their company). Independent drivers can be linked to nobody; company drivers carry `companyId` and inherit the company's agreement. Changing a template creates a new version; signed contracts never change.

Money

- All amounts are **integer paise**. `computeQuote` applies base + per-km, minimum fare, night and surge multipliers, rounding, then GST; totals always equal subtotal + GST.
- Commission = basis points on the **taxable value** (driver → company → default). Cash jobs: provider holds the cash and owes platform commission + GST; online jobs: platform owes provider net. Both appear in **Admin → Payouts & ledger**.

- Invoices: FY-sequenced numbers (`RS/2026-27/000001`), SAC code, CGST+SGST (Delhi supply) or IGST, seller/buyer snapshots.

Security

OTP sign-in (hashed codes, attempt limits, per-identifier/IP rate limits), server-side sessions (hashed tokens, revocable), role checks in layouts *and* every server action, input validation with Zod, CSP/HSTS/frame headers, markdown sanitised by construction, signed webhooks, constant-time comparisons, audit log, staff excluded from analytics, redacted OTPs in logs.

Scaling path (India-wide)

1. **Today:** single region, polling (5 s) for tracking/offers, DB-backed rate limits and cache tags — fine for tens of thousands of bookings/day on Neon + Vercel.
2. **Next:** swap polling for SSE/WebSockets (Vercel Functions support them) and move `lastLat/lastLng` writes to Redis; PostGIS for geo-matching; Neon read replica for admin reports.
3. **Multi-city:** `Area.city` already partitions coverage; add `cityId` to rate cards/settings overrides; per-city dispatch tuning.
4. **Extraction candidates:** `dispatch` (stateful, latency-sensitive) and `notifications` (queue) first — both already isolated behind service functions.